

Ping'ing XMLSec

Ping'ing XMLSec - Author not stated, blog.tint0.com.

- Published: date not stated
- Original: <<https://blog.tint0.com/2021/09/pinging-xmlsec.html>>
- Preserved from: <https://blog.tint0.com/2021/09/pinging-xmlsec.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Apache Santuario, commonly known as Apache XML Security, is a widely used library to handle XML Digital Signature and XML Encryption. It's also one of the few external libraries bundled in the JDK under the repackaged `com.sun` namespace. This post details a form of attack on the library and showcases how it could lead to heavy information leak on one of the [popular](#) Single Sign On products relying on it, PingFederate.

An attack vector on Santuario

XML Digital Signature is documented in the W3C xmldsig specs [\[\[1\]\]](#) (<https://www.w3.org/TR/2013/REC-xmldsig-core1-20130411>). A special feature was described in section 4.4.3.1: the xmldsig processing application is expected to dereference the uri in http scheme, or in other words to invoke http requests from it. Santuario implements the mechanism under *ResolverDirectHTTP*, but what's more interesting is that it resolves file uri scheme as well under *ResolverLocalFilesystem*.

At first thought it seems this mechanism could only happen in a Reference element which has the limitation that the codepath is only reachable after a valid Signature check. That in turn requires one to have a trusted private key to forge and sign the message, restricting the scenario to (kind of) post-auth. Specified at section 4.5.3 and 4.5.10, it turns out there are two more elements *KeyInfoReference* and *RetrievalMethod* that both use the same dereference mechanism. As part of the *KeyInfo* operation, they are expected to be handled before any Signature check and thus can lead to a pre-auth exploit. With this, one can embed any local file resource inside the XML DOM structure, however there's still no way yet to extract its contents.

Another special feature taking place after resource dereferencing is the Transform element, which does what its name says. There are several possible transforms but XPath and XSLT immediately stick out. With those at hands, an idea for the exploit is to construct an XPath that isolates a specific part of the target XML node, forms a conditional test on it, then construct computationally

intensive XSLT queries that take up heavy processing time inside one of the two conditional branches. With the proper timing this allows one to determine whether his XPath query is True or False and consequently form an oracle. This can be exploited to leak every XML node's contents, especially in the referenced local xml file, one part at a time.

The Transform in which case would look like:

```
...
<ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
  <ds:RetrievalMethod URI="file:/some/important/secret.xml">
    <ds:Transforms>
      <ds:Transform Algorithm="http://www.w3.org/TR/1999/REC-xslt-19991116">
        <xsl:stylesheet version="1.0" xmlns:foo="http://foo" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
          <xsl:template match="foo:bar">
            <xsl:if test="substring(./@pass,1,3)='sec'"> (1)
              <xsl:for-each select="//.">
                <xsl:for-each select="//.">
                  <xsl:for-each select="//.">
                    <xsl:for-each select="//.">
                      <xsl:for-each select="//.">
                        <a/>
                      </xsl:for-each>
                    </xsl:for-each>
                  </xsl:for-each>
                </xsl:for-each>
              </xsl:for-each>
            </xsl:if>
          </xsl:template>
        </xsl:stylesheet>
      </ds:Transform>
    </ds:Transforms>
  </ds:RetrievalMethod>
</ds:KeyInfo>
...
```

Assuming the target to extract is `<foo:bar pass="secret"/>`, the XPath at (1) tests whether the attribute value starts with 'sec' while the inner XSLT code, which was optimized from an XSLT Denial of Service payload, is meant to take about 4 seconds to process, forming quite a reliable timing oracle. Another idea for an XPath oracle is to construct an adjacent element that has an HTTP resource reference operation such that only when the XPath query satisfies can the HTTP request fire.

The biggest limitation with this is by using either XPath or XSLT, the referenced local file is required to be valid xml data. However that could already be critical when apps store their secrets in xml files.

Santuario secureValidation bypass

Santuario implements a defense in depth property named `secureValidation` and with that turned on, it refuses to load `ResolverDirectHTTP` or `ResolverLocalFilesystem`. However during handling of a `KeyInfoReference` element, the `secureValidation` property is not properly passed down to the new object so it always bears the default setting, which is off. The bug was assigned CVE-2021-40690. Although simple, with this bypass there isn't any mechanism in Santuario to protect against the above attack. Unfortunately the attack vector and how it could be exploited by default are not highlighted in the [advisory](#) from Santuario.

There are at least 3 places in Santuario where this can be exploited: the `Reference` element, the `KeyInfo` element and the `EncryptedKey` element. `KeyInfo` is usually designed to support embedded certificate and `EncryptedKey` is part of XML Encryption specs [\[\[2\]\]](#) (<https://www.w3.org/TR/2013/REC-xmlenc-core1-20130411/#sec-eg-EncryptedKey>), both are considered pre-auth.

Ping

PingIdentity's flagship `PingFederate` is a common product for organizations to implement their SSO solution. It serves as a nice demonstration as their SAML implementation relies heavily on Santuario. Naturally this does not mean it's the only product affected.

Ping went one step further and implemented a custom safeguard at `org.sourceid.common.dsig.XmlSignatureVerifier.validateRestrictions()` to enforce an allowlist of transform *Algorithm* a Signature can contain and it does not have XPath. This could be circumvented by using a `RetrievalMethod` element to reference an external document via http so when the external document is loaded, its contents would not have to go through `.validateRestrictions()`, a TOCTOU issue. Another bypass that works in `PingFederate`'s versions at least from 10.1 backwards is that when handling an attribute whose name start with `::` such as `:::Algorithm`, the inconsistency in xml node parsing from `Xmlbeans` underlying parser `Piccolo`, `PingFederate` and `Xerces` lead to a mismatch in attribute identification.

Leaking files via the XPath oracle could have worked well, however exploiting it is more straightforward in `PingFederate`. Santuario has a neat type of exception that is only thrown when handling XPath query: `XMLSecurityRuntimeException`, subclass of `Java RuntimeException`. What's special about it is that the exception message includes the current XPath node `.toString()` which always contains the full data that that node represents. And as it's an unchecked exception, it propagates all the way into upper layers of the app. A common behaviour among SAML supported apps, `PingFederate` included, is that since most [SAML bindings](#) are HTTP-based, error messages are usually caught, redirected to the log and a generic error page is returned instead. Except for SOAP-based binding which is special because it's designed to be an API-friendly interface, `PingFederate` includes additional basic error information in the SOAP response: the message fetched from `Exception.getMessage()`, which in this case is the unchecked exception received earlier. This can be considered another attack vector in addition to the XPath oracle. It has the same underlying mechanism (in that they both use resource dereferencing and transform), requires more conditions but is easier to exploit.

Following is snippet of a sample exploit via the KeyInfo element. The attribute *Id="payload"* below needs to be registered in the DOM as the [ID attribute](#), in order for it to be referenced to. To do that, one can either declare a DOCTYPE ATTLIST or rely on the application for a call to *org.w3c.dom.Element.setIdAttributeNode()*.

```
...
<ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
  <ds11:KeyInfoReference URI="#payload" xmlns:ds11="http://www.w3.org/2009/xmldsig11#" /> (1)
  <ds:X509Data/>
</ds:KeyInfo>
<ds:KeyInfo Id="payload" xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
  <ds:RetrievalMethod URI="file:/opt/out/instance/server/default/data/pingfederate-admin-user.xml">
    <ds:Transforms>
      <ds:Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" ::Algorithm="http://www.w3.org/TR/1999/R
        <ds:XPath>function-available(substring('ds:',1,3*number(
self::text() and name(parent::node())='adm:hash' )))</ds:XPath> (3)
      </ds:Transform>
      <ds:Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
    </ds:Transforms>
  </ds:RetrievalMethod>
</ds:KeyInfo>
...
```

(1) is used to bypass Santuario secureValidation, (2) to bypass PingFederate Algorithm check and the XPath at (3) is meant to throw an Exception when the current node matches the target node which in this case is when the condition `self::text() and name(parent::node())='adm:hash'` is met. The resulting SOAP response should contain the hash of PingFederate admin user in the form of an exception. On top of that, XSLT can assist to dump more, or all xml nodes in one go.

Escalating vectors

One thing might be needed to complete the chain. PingFederate puts an encryption layer on top of sensitive plaintext credentials (which could be retrieved from the xml files) with a key called PingFederate MasterKey stored in [pf.jwk](#). This file however can't be extracted via the above attack as it's not an xml. As it happens, there's also an XXE vulnerability in PingFederate with the root cause in *XmlBeansUtil.newDomNode()* plus the fact XmlBeans and PingFederate not completely sanitizing the doctype declaration. Using it to extract pf.jwk is pretty straightforward via out-of-band methods as the file is a one-line Json Web Key and does not have special characters. That was assigned CVE-2021-41770.

Most PingFederate configurations are stored in xml files, as such a very large part of information about the target SSO portal can be directly obtained via the xml disclosure attack. Other than that, there are several types of credentials we picked up during the research that may assist in direct escalation:

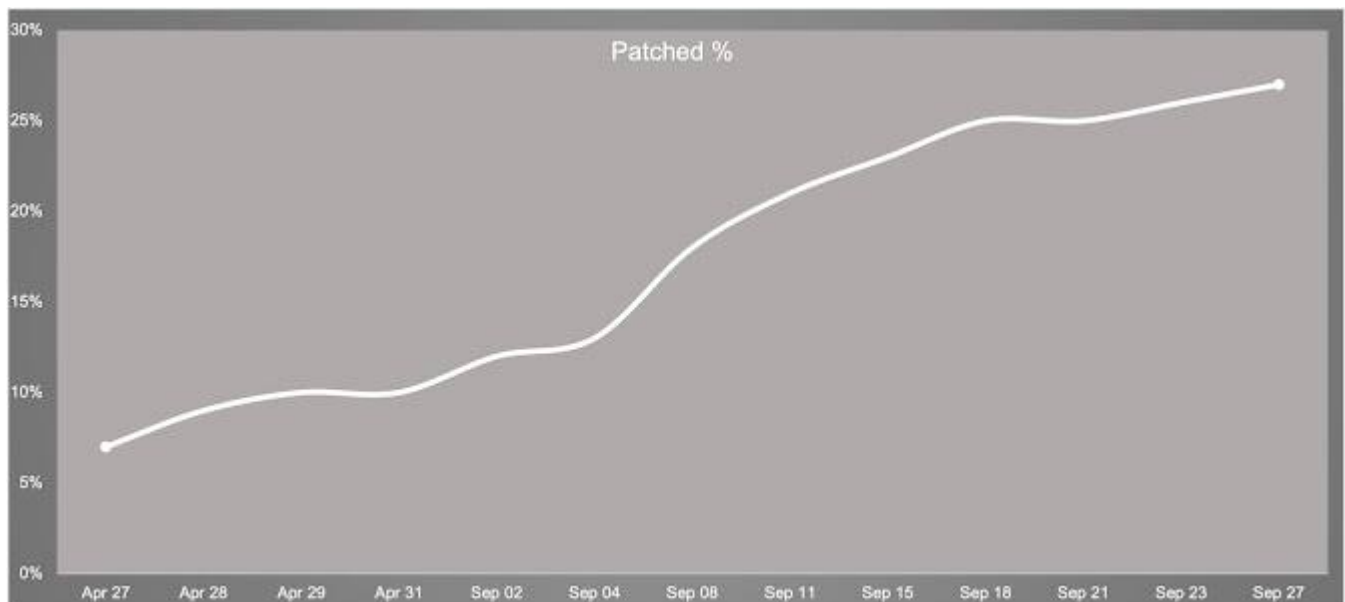
- Encrypted, reversible secrets for various configurations such as OIDC and custom adapters at `adapter-config.xml`, `password-credential-validators.xml` and `bearer-access-token-management-plugins/*.xml`
- Credentials to invoke [SCIM](#) api at `sourceid-soap-auth.xml` (switched in later versions from encrypted secrets to hashes)
- PingFederate's internal keys at `pingfederate-system-keys.xml` to forge password reset tokens for local identity profiles

The product itself is quite complex and each organization has its own unique set of configurations. Through evaluating real world targets, the biggest and most demonstrable impact by far is the disclosure of various external management API information and credentials, from which one can pivot to access and manage the company's production SSO data.

Organizations' reaction

Taking into account the size and complexity of the product it's impressive how several companies resolved this in a matter of days. These yielded maximum bounty rewards from Netflix, PayPal, and another big name who wishes to remain undisclosed. Ping also rewarded nicely for the reports.

After the patches were out, we wanted to monitor the Internet and keep track of their patching progress. The trick used here is to check for the Last-Modified header of a static file on the target (idea courtesy of Orange). While it theoretically makes sense considering a typical product update process, it's still not a guaranteed probe so there most likely will be false positives (actual patched status larger than shown).



Conclusion

In addition to the bypass fix, quite a number of changes were made to the Santuario library, essentially hardening it against this attack vector. They are filed under [SANTUARIO-572](#), [573](#), [574](#), [575](#) and [577](#). Users should upgrade to version 2.2.3 or 2.1.7, but preferably 2.3.0 when it comes out as it incorporates more hardening. As for Ping, they released the patches nearly 2 months ago

for all their product versions to address the issues. An advisory is still not yet released, but I was told it's underway.

Finally, props to all parties for their prompt responses, additionally to Ping for an uncommonly well written product that makes a nice challenge.

[1] <https://www.w3.org/TR/2013/REC-xmlsig-core1-20130411> [2] <https://www.w3.org/TR/2013/REC-xmlenc-core1-20130411/#sec-eg-EncryptedKey>

Archived from <https://blog.tint0.com/2021/09/pinging-xmlsec.html>. Rendered offline from the local Markdown copy.